

DOCKER • DJANGO • CELERY • POSTGRESQL • REDIS

Library Management System

Docker Configuration & Project Setup Guide

A complete, step-by-step reference for containerizing the Django Library Management System, plus a full Docker command cheat sheet for day-to-day development.

Table of Contents

Part 1 — Project Setup Guide

- 1. Overview of the Stack
- 2. Prerequisites
- 3. Understanding the `Dockerfile`
- 4. Environment Variables & the `.env` File
- 5. Understanding & Extending `docker-compose.yml`
- 6. Fixing the Celery Broker URL for Docker Networking
- 7. Building and Starting the Stack
- 8. Running Migrations & Creating a Superuser
- 9. Verifying Everything Works
- 10. Security Notes on Current Configuration
- 11. Troubleshooting Table

Part 2 — Docker Command Reference

- Docker Normal Commands
- Docker Execution Commands
- Docker Django Execution Commands
- Docker Django Log Commands
- Docker PostgreSQL Login Commands
- Docker PostgreSQL Log Commands
- Docker Redis Commands
- Docker Celery Execution Commands
- Docker Celery Log Commands
- Full Reference Table
- Docker All Known Commands (Extra Reference)

PART 1

Project Setup Guide

1. Overview of the Stack

The Library Management System is a Django project made of four moving pieces once it runs in Docker:

Service	Role	Image / Source
web	Django app (the <code>library_system</code> project + <code>members</code> app)	Built from your <code>Dockerfile</code>
db	PostgreSQL database	<code>postgres:15</code>
redis	Message broker + result backend for Celery	<code>redis:7-alpine</code> (not yet in your compose file — added in Step 5)
celery	Runs background tasks (e.g. <code>members/tasks.py</code> , fine maintenance)	Same image as web , different command

Your current `docker-compose.yml` only defines **db** and **web**. Your project code already depends on Celery and Redis (`library_system/celery.py` , `CELERY_BEAT_SCHEDULE` in `settings.py`), so this guide adds the missing **redis** and **celery** services in Step 5 — otherwise background tasks and fine maintenance won't run inside Docker.

2. Prerequisites

- Docker Desktop (Windows/Mac) or Docker Engine + Compose plugin (Linux) installed — verify with `docker --version` and `docker compose version`
- Your project folder `library-management-system/` containing `Dockerfile` , `docker-compose.yml` , `requirements.txt` and `manage.py`
- Ports **8000**, **5432**, and **6379** free on your host machine

3. Understanding the Dockerfile

Here is your Dockerfile with an explanation of every instruction:

```
# Base image: slim Python 3.11
FROM python:3.11-slim
WORKDIR /app

# System packages needed to compile psycopg2 (PostgreSQL driver)
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential libpq-dev gcc \
    && rm -rf /var/lib/apt/lists/*

# Install Python deps first (better layer caching -> faster rebuilds)
COPY requirements.txt .
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

# Now copy the rest of the project code
COPY . .
EXPOSE 8000
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

Line	Purpose
<code>FROM python:3.11-slim</code>	Small Debian-based image with Python 3.11 preinstalled
<code>WORKDIR /app</code>	All subsequent commands run relative to <code>/app</code> inside the container
<code>build-essential libpq-dev gcc</code>	Needed to build <code>psycopg2</code> against PostgreSQL headers
<code>COPY requirements.txt .</code> before <code>COPY . .</code>	Docker layer caching: dependencies only reinstall when <code>requirements.txt</code> changes, not on every code edit
<code>EXPOSE 8000</code>	Documents the port Django listens on (doesn't publish it — that's done in <code>docker-compose.yml</code>)
<code>CMD [...]</code>	Default command; overridden by the <code>command:</code> line in <code>docker-compose.yml</code>

Good news: this Dockerfile is already solid for development. No changes required for the setup below.

4. Environment Variables & the .env File

Your `settings.py` calls `load_dotenv(BASE_DIR / ".env")` and reads `SECRET_KEY` , `DEBUG` , `ALLOWED_HOSTS` , and all five `DB_*` values via

`os.getenv()` . That means a `.env` file is **required** at the project root — without it, `SECRET_KEY` will be `None` and Django will refuse to start.

1 Create `library-management-system/.env` :

```
# --- Django core ---
SECRET_KEY=replace-this-with-a-long-random-string
DEBUG=True
ALLOWED_HOSTS=localhost,127.0.0.1

# --- Database (must match docker-compose db service) ---
DB_ENGINE=django.db.backends.postgresql
DB_NAME=library_db
DB_USER=library_user
DB_PASSWORD=library_pass
DB_HOST=db
DB_PORT=5432

# --- Email (move the hardcoded values out of settings.py, see Section 10) ---
EMAIL_HOST_USER=your-email@gmail.com
EMAIL_HOST_PASSWORD=your-gmail-app-password

# --- Celery / Redis (used once you add the redis service in Step 5) ---
CELERY_BROKER_URL=redis://redis:6379/0
CELERY_RESULT_BACKEND=redis://redis:6379/0
```

Note: generate a real `SECRET_KEY` rather than typing text by hand. Inside any Django shell: `python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"`

2 Add `.env` to `.gitignore` so it's never committed:

```
echo ".env" >> .gitignore
```

Why `DB_HOST=db` and not `localhost` : inside Docker's internal network, containers reach each other by **service name** (as defined in `docker-compose.yml`), not `localhost` . `db` is the hostname Docker Compose automatically assigns to the PostgreSQL container on the shared network.

5. Understanding & Extending `docker-compose.yml`

Your current file defines two services. Below it is explained, then extended with `redis` and `celery` so the whole stack (including background tasks) runs together.

5.1 Your current configuration, annotated

```
services:
  db:
    image: postgres:15
    restart: always
    environment:
      POSTGRES_DB: library_db           # must match DB_NAME in .env
      POSTGRES_USER: library_user      # must match DB_USER in .env
      POSTGRES_PASSWORD: library_pass  # must match DB_PASSWORD in .env
    volumes:
      - postgres_data:/var/lib/postgresql/data  # persists DB data across restarts
    ports:
      - "5432:5432"                          # host:container - lets you connect with a DB GUI tool

  web:
    build: .                                # builds from your Dockerfile
    command: python manage.py runserver 0.0.0.0:8000
    volumes:
      - ../app                               # live code reload without rebuilding the image
    ports:
      - "8000:8000"
    depends_on:
      - db                                    # waits for db container to start (not for DB readiness!)
    environment:
      DB_NAME: library_db
      DB_USER: library_user
      DB_PASSWORD: library_pass
      DB_HOST: db
```

```
DB_PORT: 5432
```

```
volumes:  
  postgres_data:
```

Gap: the `web` service only passes DB variables. `SECRET_KEY`, `DEBUG`, `ALLOWED_HOSTS`, `DB_ENGINE`, and email settings are never injected into the container, so it's cleaner to load the whole `.env` file with `env_file:` instead of listing variables one by one.

5.2 Recommended, complete docker-compose.yml

Replace the contents of `docker-compose.yml` with this version, which keeps your original two services intact and adds `redis` and `celery`:

```
services:  
  db:  
    image: postgres:15  
    restart: always  
    environment:  
      POSTGRES_DB: library_db  
      POSTGRES_USER: library_user  
      POSTGRES_PASSWORD: library_pass  
    volumes:  
      - postgres_data:/var/lib/postgresql/data  
    ports:  
      - "5432:5432"  
    healthcheck:  
      test: ["CMD-SHELL", "pg_isready -U library_user -d library_db"]  
      interval: 5s  
      timeout: 5s  
      retries: 5  
  
  redis:  
    image: redis:7-alpine  
    restart: always  
    ports:  
      - "6379:6379"  
    volumes:  
      - redis_data:/data  
  
  web:  
    build: .  
    command: python manage.py runserver 0.0.0.0:8000  
    volumes:  
      - ./app  
    ports:  
      - "8000:8000"  
    env_file:  
      - .env  
    depends_on:  
      db:  
        condition: service_healthy  
      redis:  
        condition: service_started  
  
  celery:  
    build: .  
    command: celery -A library_system worker --loglevel=info  
    volumes:  
      - ./app  
    env_file:  
      - .env  
    depends_on:  
      - web  
      - redis  
      - db  
  
  celery-beat:  
    build: .  
    command: celery -A library_system beat --loglevel=info  
    volumes:  
      - ./app  
    env_file:  
      - .env  
    depends_on:
```

```

- web
- redis
- db

volumes:
  postgres_data:
  redis_data:

```

Addition	Why
<code>healthcheck</code> on <code>db</code>	<code>depends_on</code> only waits for the container to <i>start</i> , not for Postgres to be ready to accept connections. The healthcheck fixes the race condition where <code>web</code> starts before the DB is actually ready.
<code>redis</code> service	Required broker/backend for Celery — matches <code>CELERY_BROKER_URL</code>
<code>env_file:</code> <code>.env</code>	Loads every variable from <code>.env</code> into the container in one line instead of duplicating each one under <code>environment:</code>
<code>celery</code> service	Runs the worker that executes tasks defined in <code>members/tasks.py</code>
<code>celery-beat</code> service	Runs the scheduler that triggers <code>CELERY_BEAT_SCHEDULE</code> entries (e.g. <code>fine-maintenance-midnight</code>) on time

6. Fixing the Celery Broker URL for Docker Networking

Your `settings.py` currently hardcodes:

```

CELERY_BROKER_URL = 'redis://localhost:6379/0'
CELERY_RESULT_BACKEND = 'redis://localhost:6379/0'

```

Inside a container, `localhost` refers to the `web` / `celery` container itself — not the `redis` container — so Celery will fail to connect. Update these two lines to read from the environment instead:

```

CELERY_BROKER_URL = os.getenv("CELERY_BROKER_URL", "redis://redis:6379/0")
CELERY_RESULT_BACKEND = os.getenv("CELERY_RESULT_BACKEND", "redis://redis:6379/0")

```

This keeps things working both in Docker (where the `.env` value `redis://redis:6379/0` is used) and on your bare-metal `menv` setup (where the default falls back to `localhost`).

7. Building and Starting the Stack

- 1 From the project root (where `docker-compose.yml` lives):

```

cd ~/library_system/library-management-system
docker compose build

```

- 2 Start everything in the background:

```

docker compose up -d

```

- 3 Check that all four containers are healthy:

```

docker compose ps

```

You should see `db`, `redis`, `web`, `celery`, and `celery-beat` all in a running/healthy state.

8. Running Migrations & Creating a Superuser

Your `members` app has 30 migrations — apply them inside the running `web` container:

```

docker compose exec web python manage.py migrate
docker compose exec web python manage.py createsuperuser

```

Since `./app` is mounted as a volume, this only needs to be re-run when you add *new* migrations — existing data in the `postgres_data` volume persists across `docker compose down` / `up` cycles (but not `down -v`).

9. Verifying Everything Works

Check	How
Django is serving	Open <code>http://localhost:8000</code> in a browser
Admin panel loads	Open <code>http://localhost:8000/admin</code> , log in with the superuser you created
Database connected	<code>docker compose exec web python manage.py showmigrations</code> runs without connection errors
Redis reachable	<code>docker compose exec redis redis-cli ping</code> → should return <code>PONG</code>
Celery worker connected	<code>docker compose logs celery</code> → look for <code>celery@... ready</code>
Beat schedule loaded	<code>docker compose logs celery-beat</code> → look for <code>Scheduler: Sending due task fine-maintenance-midnight</code>

10. Security Notes on Current Configuration

Hardcoded email credentials: `settings.py` currently has a real Gmail address and app password written directly in the file (`EMAIL_HOST_USER` , `EMAIL_HOST_PASSWORD`). This should follow the same pattern as your DB settings:

```
EMAIL_HOST_USER = os.getenv("EMAIL_HOST_USER")
EMAIL_HOST_PASSWORD = os.getenv("EMAIL_HOST_PASSWORD")
```

with the actual values only living in `.env` (which is git-ignored). If this file has ever been committed or shared, revoke that Gmail app password from your Google Account → Security → App Passwords, and generate a new one.

DEBUG in production: keep `DEBUG=True` only for local Docker development. Set `DEBUG=False` and a real, non-wildcard `ALLOWED_HOSTS` before deploying anywhere public.

11. Troubleshooting Table

Symptom	Likely Cause	Fix
<code>web</code> exits immediately	Missing/invalid <code>.env</code> , so <code>SECRET_KEY</code> is <code>None</code>	Confirm <code>.env</code> exists at project root and <code>env_file: .env</code> is set on <code>web</code>
<code>django.db.utils.OperationalError: could not connect to server</code>	<code>web</code> started before Postgres was ready	Use the <code>healthcheck</code> + <code>condition: service_healthy</code> shown in Section 5.2
Celery: <code>Cannot connect to redis://localhost:6379/0</code>	Broker URL still hardcoded to <code>localhost</code>	Apply the fix in Section 6
Port 5432 already in use	A local (non-Docker) Postgres is already running	Stop the local Postgres service, or change the host port mapping to <code>"5433:5432"</code>
Code changes not reflected	Stale image, or <code>volumes: ../app</code> missing	<code>docker compose up -d --build</code>

PART 2

Docker Command Reference

Docker Normal Commands

```
docker --version          # check installed Docker version
docker info               # system-wide Docker info
docker ps                 # list running containers
docker ps -a              # list all containers (incl. stopped)
docker images              # list local images
docker rm <container>     # remove a stopped container
docker rmi <image>         # remove an image
docker stop <container>   # gracefully stop a container
docker start <container>  # start a stopped container
docker restart <container> # restart a container
docker inspect <container|image> # full JSON metadata
docker volume ls          # list volumes
docker network ls         # list networks
docker system prune       # remove unused containers/images/networks
docker system df          # disk usage summary
```

Docker Execution Commands

```
docker compose build      # build images from docker-compose.yml
docker compose up         # start stack, logs attached to terminal
docker compose up -d      # start stack in background (detached)
docker compose up -d --build # rebuild images then start
docker compose down       # stop and remove containers/network
docker compose down -v    # also remove named volumes (deletes DB data!)
docker compose stop       # stop containers, keep them
docker compose start      # start previously-stopped containers
docker compose restart    # restart all services
docker compose restart web # restart a single service
docker compose ps         # status of services in this project
docker compose exec web bash # shell into a running service
docker compose run --rm web bash # one-off container, removed after exit
docker compose config     # validate & print resolved compose file
```

Docker Django Execution Commands

```
docker compose exec web python manage.py migrate
docker compose exec web python manage.py makemigrations
docker compose exec web python manage.py makemigrations members
docker compose exec web python manage.py createsuperuser
docker compose exec web python manage.py collectstatic --noinput
docker compose exec web python manage.py shell
docker compose exec web python manage.py showmigrations
docker compose exec web python manage.py test
docker compose exec web python manage.py dbshell
docker compose exec web bash
docker compose run --rm web python manage.py migrate # run once, no lingering container
```

Docker Django Log Commands

```
docker compose logs web          # dump all web logs so far
docker compose logs -f web       # follow (live tail) web logs
docker compose logs --tail=100 web # last 100 lines only
docker compose logs -f --tail=50 web # follow, starting from last 50 lines
docker logs <web_container_id>   # same, using raw container id/name
docker logs -f <web_container_id>
```

Docker PostgreSQL Login Commands (username / password variants)

```
# Standard login (will prompt for library_user's password: library_pass)
docker compose exec db psql -U library_user -d library_db

# Explicit interactive TTY flag (useful outside compose context)
docker compose exec -it db psql -U library_user -d library_db

# Using raw container name instead of compose service name
docker exec -it library-management-system-db-1 psql -U library_user -d library_db

# Passing the password inline, skips the prompt
PGPASSWORD=library_pass docker compose exec db psql -U library_user -d library_db

# Login as the postgres superuser (only exists if you set POSTGRES_USER=postgres)
docker compose exec db psql -U postgres

# From your HOST machine (requires local psql client + port 5432 published)
psql -h localhost -p 5432 -U library_user -d library_db
```

Docker PostgreSQL Log Commands

```
docker compose logs db                # all db logs
docker compose logs -f db             # follow db logs live
docker compose logs --tail=100 db     # last 100 lines
docker compose logs --since=10m db    # logs from the last 10 minutes
docker exec -it <db_container> cat /var/log/postgresql/postgresql.log # if file logging enabled
```

Docker Redis Commands

```
docker compose exec redis redis-cli    # open interactive redis-cli shell
docker compose exec redis redis-cli ping # health check -> PONG
docker compose exec redis redis-cli keys '*' # list all keys (dev/debug only)
docker compose exec redis redis-cli monitor # live stream of every command hitting redis
docker compose exec redis redis-cli info   # server stats
docker compose exec redis redis-cli flushall # wipe all keys - use with care
docker compose logs -f redis              # follow redis logs
```

Docker Celery Execution Commands

```
# Start a worker manually (already automated via the celery service in Section 5.2)
docker compose exec celery celery -A library_system worker --loglevel=info

# One-off worker run without a persistent service
docker compose run --rm celery celery -A library_system worker --loglevel=info

# Check status of active workers
docker compose exec celery celery -A library_system status

# Inspect currently active/reserved/scheduled tasks
docker compose exec celery celery -A library_system inspect active
docker compose exec celery celery -A library_system inspect scheduled

# Restart the worker after code changes
docker compose restart celery

# Restart the beat scheduler
docker compose restart celery-beat
```

Docker Celery Log Commands

```
# Main one - confirms a task ran: you'll see "Task succeeded" or a full traceback
docker compose logs -f celery

docker compose logs celery                # all logs so far, no follow
docker compose logs --tail=200 celery    # last 200 lines
```

```
docker compose logs -f celery-beat           # follow the scheduler's logs
docker compose logs -f celery celery-beat    # follow both worker and beat together
```

Full Reference Table

Category	Command	Purpose
Normal	<code>docker ps -a</code>	List all containers
Normal	<code>docker system prune</code>	Clean unused resources
Execution	<code>docker compose up -d --build</code>	Rebuild & start stack
Execution	<code>docker compose down -v</code>	Stop & wipe volumes
Django	<code>docker compose exec web python manage.py migrate</code>	Apply migrations
Django	<code>docker compose exec web python manage.py createsuperuser</code>	Create admin user
Django Logs	<code>docker compose logs -f web</code>	Live Django logs
PostgreSQL	<code>docker compose exec db psql -U library_user -d library_db</code>	Open DB shell
PostgreSQL Logs	<code>docker compose logs -f db</code>	Live DB logs
Redis	<code>docker compose exec redis redis-cli ping</code>	Check Redis is alive
Celery	<code>docker compose exec celery celery -A library_system status</code>	Worker status
Celery Logs	<code>docker compose logs -f celery</code>	Confirm task success/failure

Docker All Known Commands (Extra Reference)

```
# Image & build
docker build -t <name> .
docker tag <image> <repo>:<tag>
docker push <repo>:<tag>
docker pull <image>
docker save -o image.tar <image>
docker load -i image.tar
docker history <image>

# Running & inspecting containers
docker run -d -p 8000:8000 <image>
docker exec -it <container> bash
docker cp <container>:/app/file.txt ./file.txt
docker top <container>
docker stats
docker diff <container>

# Networks & volumes
docker network create <name>
docker network inspect <name>
docker volume create <name>
docker volume inspect <name>
docker volume rm <name>

# Compose extras
docker compose pull
docker compose logs --since=1h
docker compose top
docker compose events
docker compose port web 8000
docker compose cp web:/app/somefile.txt ./somefile.txt

# Cleanup
docker builder prune
docker image prune -a
docker container prune
docker volume prune
docker network prune
```

Quick start recap: `docker compose build` → `docker compose up -d` → `docker compose exec web python manage.py migrate` → `docker compose exec web python manage.py createsuperuser` → open `http://localhost:8000` .